

The Incremental Funding Method: Data-Driven Software Development

Mark Denne, *Sun Microsystems Inc.*

Jane Cleland-Huang, *DePaul University*

In today's competitive environment, software development projects don't get funded unless they return clearly defined value to the business. Demands for shorter investment periods, faster time-to-market, and increased agility require new, radical software development approaches. These approaches must draw on the expertise of both software architects and financial stakeholders and open the traditional black box of software development to rigorous financial analysis. We can accomplish this

only by positioning software development as a value-creation activity in which business analysis is integral.¹

The Incremental Funding Method is a financially informed approach to software development.² IFM maximizes returns by delivering functionality in "chunks" of customer-valued features, carefully sequenced to optimize the project's *net present value* (NPV). We derived the IFM concepts from several years' experience in winning competitive contracts for large-systems integration and application development projects.

Instead of focusing exclusively on controlling risk and cost, IFM also addresses "time to value" (that is, the rate at which value is returned to the customer). It can work in conjunction with any iterative software development process such as the Rational Unified Process, Extreme Programming, Feature-Driven Development, or Crystal methodologies.

The importance of cash flow

Consider the hypothetical software development project in Table 1. In this example, cash flow is analyzed quarterly over a two-year period. Development and testing lasts one year. Following the product's release, it generates US\$800K revenue per quarter. Costs include some early capital outlays associated with buying the hardware and software needed for the development work. A technology refresh cycle

IFM analyzes and sequences feature delivery to maximize net present value and to provide insight on how development decisions affect other financial metrics.

Table 1

Cash flow analysis for a software development project using traditional feature delivery (US\$ thousands)

	Period								
	1	2	3	4	5	6	7	8	Total
Revenue					800	800	800	800	3,200
Hardware	-200	-10	-10	-10	-100	-10	-10	-10	-360
Software	-200	-10	-10	-10	-10	-10	-10	-10	-270
Headcount	-300	-300	-300	-300	-300	-100	-50	-50	-1,700
Data center	-50	-50	-50	-50	-40	-10	-10	-10	-270
Marketing	0	0	0	-50	-100	-100	-100	-50	-400
Cash	-750	-370	-370	-420	250	570	620	670	200
Investment	-750	-370	-370	-420					-1,910
ROI									10%

Table 2

Cash flow analysis for the same project using incremental feature delivery (US\$ thousands)

	Period								
	1	2	3	4	5	6	7	8	Total
Feature 1		200	200	200	200	200	200	200	
Feature 2			200	200	200	200	200	200	
Feature 3				200	200	200	200	200	
Feature 4					200	200	200	200	
Revenue	0	200	400	600	800	800	800	800	4,400
Hardware	-300	-15	-15	-150	-15	-15	-15	-15	-540
Software	-300	-15	-15	-15	-15	-15	-15	-15	-405
Headcount	-400	-400	-400	-300	-300	-100	-50	-50	-2,000
Data center	-50	-50	-50	-50	-50	-10	-10	-10	-280
Marketing	-50	-100	-100	-100	-100	-100	-100	-50	-700
Cash	-1,100	-380	-180	-15	320	560	610	660	475
Investment	-1,100	-380	-180	-15					-1,675
ROI									28%

Table 3

A DCF (discounted cash flow) comparison of the traditional delivery approach vs. the incremental delivery approach (US\$ thousands)

	Period								
Delivery	1	2	3	4	5	6	7	8	NPV
Traditional									
Cash	-750	-370	-370	-420	250	570	620	670	200
DCF @ 2.5%	-732	-352	-344	-380	221	492	522	550	-24
Incremental									
Cash	-1,100	-380	-180	-15	320	560	610	660	475
DCF @ 2.5%	-1,073	-362	-167	-14	283	483	513	542	205

NPV: Net present value

after the first year updates the development hardware. In addition to the capital costs, the project has recurring support costs for the hardware and software and operational costs related to personnel, data center facilities, and marketing. For the purposes of this analysis, we assume that the revenue flow is constant and that there are no updates during the year following release. The project requires initial funding of \$1.91M, and it returns \$200K in undiscounted cash by the analysis period's end. The traditional return on investment is 10 percent over two years.

Now consider what happens if we partition the application into four equally valued features developed and released independently during the project's first four quarters. Once delivered, each feature generates revenue at a rate of one-quarter of the project's total revenue. The project incurs several incremental costs to facilitate the early release of features. The projected hardware and software costs increase by 50 percent. The project also sustains costs for packaging, marketing, and releasing these features, along with additional headcount costs due to the additional testing. Ultimately, however, the application's development lasts the same amount of time (one year). Table 2 shows the outcome.

The incremental project has three basic advantages over the traditional project:

- The early release of features enhances the total revenue from \$3.2M to \$4.4M.
- The investment decreases from \$1.91M to \$1.675M.
- The ROI over two years increases from 10 percent to 28 percent.

Although such comparisons provide useful insights into the differences between the two approaches, metrics such as ROI and net cash fail to tell us if a project is actually worth funding. To determine this, we need to consider how time affects the cash flow. We do this by adjusting each of the projected cash positions by an assumed discount rate to create a *discounted cash flow*. The sum of the DCF is the project's NPV. For this example, we've applied a rate of 2.5 percent per quarter—equivalent to just over 10 percent per year. (In this analysis, we follow the convention used in several TCO [total cost of ownership] tools of assuming that the costs and revenues are incurred at each period's end.)

Table 3 shows that in this example the use

of traditional feature delivery results in a negative NPV, indicating that the business is drained of value if it undertakes the project, despite the positive ROI. In contrast, in this case, incremental feature delivery represents a positive NPV of \$205K. Figure 1 shows a generic DCF for a successful software project with a positive NPV. This defines some important financial metrics such as the funding cost, the breakeven time, and the length of the repayment period, which we consider in the next section.

The Incremental Funding Method

IFM decomposes the system to be developed into units of functionality known as *minimum marketable features*. MMFs are small, self-contained features that can be delivered quickly and that provide market value to the customer.²

Deconstructing a system in this way isn't new. Tom Gilb, in his 1988 book *Principles of Software Engineering Management*, mentions the IBM Federal Systems Division's experience with the Light Airborne Multi-Purpose System. LAMPS was a 200 person-year project that was delivered successfully over four years in 45 incremental deliveries.³ More recently, both the agile and traditional development communities have emphasized the value of delivering early a system's executable functioning parts.⁴⁻⁶ In the same vein, the Standish Group's Chaos Report identified small project size as one of the most significant factors in project success.⁷ By decomposing a project into MMFs that can be managed as "miniprojects," IFM increases the likelihood of success for large-scale development efforts.

An MMF's value is typically measured in terms of both tangible and intangible factors such as revenue generation, cost savings, competitive differentiation, brand-name projection, and enhanced customer loyalty. Developers, business stakeholders, marketers, and customers work together to identify the MMFs. They compose the MMFs either through a bottom-up approach that clusters low-level requirements or user stories into meaningful functional groups, or through a top-down approach that decomposes system-level functions.

For example, MMFs for a hypothetical online travel agency application might include flight, hotel, and car reservations; flight insurance; facilities that provide special offers and travel bulletins to registered customers; and a

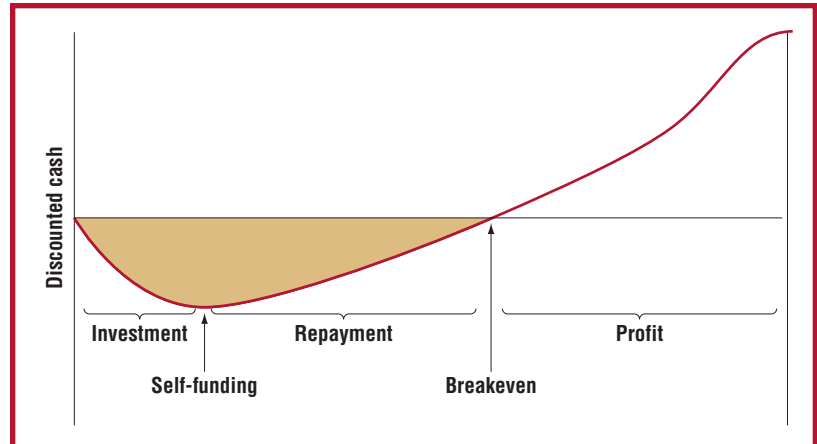


Figure 1. A generic discounted cash flow for a successful software project with a positive net present value.

static HTML Web site that markets tours (see Figure 2).

Incremental architecture

In most cases, MMFs can be deployed only after the underlying architectural infrastructure has been developed. However, if the entire architecture is constructed up front, development of revenue-generating functionality will be deferred until later, thereby reducing the project's NPV.

Although architecture has many definitions,⁸⁻⁹ they all assume that an architecture defines a system's major components or elements, and their interrelationships. So, we can

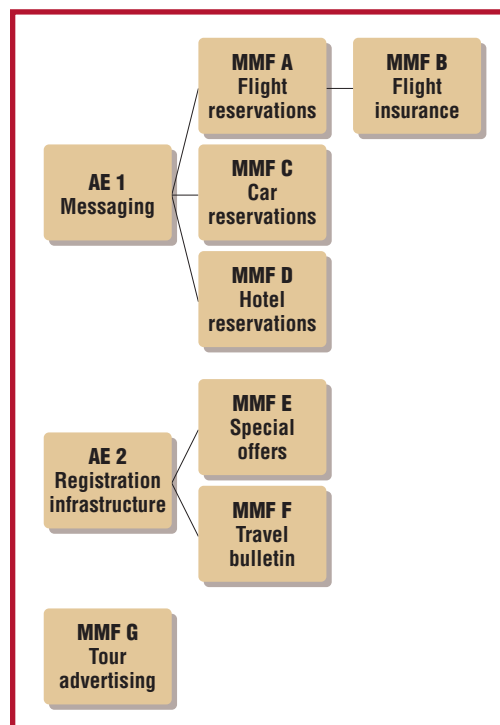


Figure 2. Business dependencies between elements in a hypothetical online travel agency application. Elements consist of minimum marketable features and architectural elements.

In most applications, business- and technology-related dependencies exist between individual elements.

decompose a system's physical architecture into deliverable elements in much the same way that we can deconstruct its functionality into MMFs. We coin the term *architectural element* to describe these elements. Our travel agency example has two AEs (see Figure 2):

- *Messaging* provides secure communications with the data store.
- *Registration infrastructure* registers users to facilitate subsequent one-to-one marketing.

Costs and returns

Once the customers, developers, and business stakeholders have identified the MMFs and AEs, the costs and projected revenues are analyzed over the number of periods established by the business. This requires close collaboration between various stakeholders. Developers estimate the cost and effort involved in developing each MMF and AE (collectively referred to as “elements” from this point on). At the same time, the business stakeholders estimate each MMF's revenue. This, of course, is easier in organizations that track statistical information about their products and clientele, but projections based on any type of previous experience are possible. In most cases, developers have neither the business knowledge nor skills needed to make these types of revenue projections.

Table 4 depicts the resulting quarterly costs and revenues for a project with a four-year analysis period. As an example, consider MMF E, which manages special offers for registered customers. This feature is projected to cost \$80K to develop over one period, and once delivered will generate revenue through increased sales to prior customers. Its projected initial revenue is \$25K per quarter, with a gradual ramp-up to \$95K per quarter by the end of the fourth year. We've already adjusted the projected revenues according to the probability and impact of risk. For example, we reduce a base revenue projection of \$20K by \$2.5K to \$17.5K, as a result of an identified 50 percent market risk of losing one-quarter of the revenue ($\$20K \times 0.25 \times 0.5 = \$2.5K$). Projected revenues can also incorporate intangible values.² Because AE's don't generate revenue, we analyze them in terms of costs only.

The projections can be updated throughout the project to reflect the most current infor-

mation available and then fed back into the sequencing process. Thus, IFM provides the flexibility needed to respond to change. This lets stakeholders keep their options open throughout the project, which can increase the project's flexibility and potential value.^{5,10}

Precursors

In most applications, business- and technology-related dependencies exist between individual elements. In the travel agency example (see Figure 2), AE 1 is a precursor to MMFs A, C, and D, MMF A is a precursor to MMF B, AE 2 is a precursor to MMFs E and F, and MMF G has no precursors. Such dependencies constrain the order in which elements can be developed, thus limiting a project's sequencing options.

Analyzing time-adjusted delivery options

Sequencing is needed because resource limitations and dependencies between elements typically make it impractical to develop all the elements during a single period. To evaluate alternate sequencing options, we must first calculate each element's NPV for each feasible period. Table 5 shows the *sequence-adjusted NPVs* for the travel agency project, calculated at a discount of 2.5 percent per quarter. For example, if MMF C is developed in Period 1, its SANPV is \$266K; if it's developed in Period 5, its SANPV decreases to \$145K.

The IFM heuristic

In small projects with few elements, it's clearly possible to calculate each feasible sequence's NPV to categorically determine the best sequence. (We've built an analysis tool that can successfully calculate sequences of up to 13 MMFs.) However, as the number of feasible delivery sequences grows exponentially with the number of elements, this approach quickly becomes impractical. To address this problem, we introduce the *IFM heuristic*.

The IFM heuristic views sequencing options as *strands*—sequences of elements linked by precursor relationships. Individual elements are special cases of strands. We decompose a string of dependent MMFs such as $M_1 \leftarrow M_2 \leftarrow M_3$ into the strands M_1 , M_1M_2 , M_2M_3 , and $M_1M_2M_3$. Similarly, a precursor relationship such as $M_1 \leftarrow M_2$ and $M_1 \leftarrow M_3$ results in strands M_1 , M_1M_2 , and M_1M_3 . For elements that take more than one period to de-

Table 4**Estimated cost and revenue for the elements (minimum marketable features and architectural elements) in the online travel agency example (US\$ thousands)**

Element	Period															
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
AE 1	-200	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
MMF A	-75	-75	60	65	70	75	80	85	90	95	100	105	110	115	120	125
MMF B	-50	6	9	10	11	11	12	13	14	14	15	16	17	17	18	19
MMF C	-50	13	15	17	19	21	23	25	27	29	31	33	35	37	39	41
MMF D	-50	10	12	14	16	18	20	22	24	26	28	30	32	34	36	38
AE 2	-110	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
MMF E	-80	25	28	30	33	37	40	44	49	54	59	65	71	78	86	95
MMF F	-75	15	20	25	30	35	35	35	35	35	35	35	35	35	35	35
MMF G	-75	25	25	25	25	25	25	25	25	25	25	25	25	25	25	25

Table 5**Sequence-adjusted net present value for the elements in the online travel agency example (US\$ thousands)**

Element	Period															
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
AE 1	-195	-190	-186	-181	-177	-172	-168	-164	-160	-156	-152	-149	-145	-142	-138	-135
MMF A	862	759	662	570	484	403	328	257	192	131	75	24	-22	-64	-102	-51
MMF B	109	94	80	67	54	42	31	21	12	3	-5	-12	-19	-25	-30	-34
MMF C	266	233	202	172	145	120	97	75	56	38	22	7	-5	-16	-26	-34
MMF D	230	199	171	144	120	97	76	57	40	25	11	-1	-11	-20	-28	-34
AE 2	-107	-105	-102	-100	-97	-95	-93	-90	-88	-86	-84	-82	-80	-78	-76	-74
MMF E	535	459	391	331	276	226	182	142	107	75	47	21	-1	-20	-38	-54
MMF F	303	273	243	214	186	158	132	105	80	55	30	7	-13	-29	-42	-51
MMF G	229	207	185	164	144	124	105	86	67	49	31	14	-3	-19	-35	-51

velop, our notation uses a period (“.”) to represent an additional development period. For example, MMF A, which takes two periods to develop, would be “A.” So, the travel agency example produces strands such as 1A., 1A.B, A.B, 1C, 1D, 2E, 2F, and G.

Next, we calculate each strand’s SANPV, representing the combined NPV for all the strand’s elements if construction starts in a given period. For example, we calculate the SANPV for 1A.B by summing the individual SANPVs of the strand’s elements:

$$\begin{aligned}
 & \text{SANPV}(1A.B, \text{Period } 1) \\
 &= \text{SANPV}(1, \text{Period } 1) \\
 &\quad + \text{SANPV}(A, \text{Period } 2) \\
 &\quad + \text{SANPV}(B, \text{Period } 4) \\
 &= -\$195K + \$759K + \$67K \\
 &= \$631K
 \end{aligned}$$

Although SANPVs provide the basis for comparing the values of specific development strands, they don’t take into account the critical factor of strand duration. For example, a strand that delivers \$100K in two development periods is preferable to one that delivers \$100K in three periods, because the shorter strand is completed more quickly and leaves additional project time for developing other valuable features. So, during sequencing, we negatively weight the strands according to their lengths, and use the weighted SANPVs (WSANPVs) to drive the selection process. IFM applies this weighting formula:

$$\text{WSANPV} = \text{SANPV} \times (1 - WF \times (P - 1)),$$

where WF is the weighting factor and P is the number of periods needed to develop the strand.

Table 6

The weighted SANPV for strands for Periods 1, 2, and 4.* The bold WSANPV numbers are the selected values.

Available strands	SANPV	WSANPV
Period 1		
1	-195	-195
1A.	564	395
1A.B	631	347
1C	38	32
1D	4	4
2E	352	299
2F	165	141
G	229	229
Period 2		
A.	759	645
A.B	826	578
C	233	233
D	199	199
2E	287	244
2F	138	118
G	207	207
Period 4		
B	67	67
C	172	172
D	144	144
2E	176	150
2F	86	73
G	164	164

* We skipped Period 3 because MMF A, which was selected in Period 2, took two periods to develop.

Empirical results (which we describe later) suggest an optimal weighting factor of 15 percent for projects of this size. For the travel agency example, we calculate the WSANPV of **1A.B** for Period 1 as

$$\begin{aligned}
 &\text{WSANPV}(\mathbf{1A.B}, \text{Period 1}) \\
 &= \$631\text{K} \times (1 - (0.15 \times (4 - 1))) \\
 &= \$347\text{K}.
 \end{aligned}$$

Table 6 depicts the WSANPVs for strands available for selection in Periods 1, 2, and 4.

How the heuristic operates. First, the IFM heuristic examines the available strands for Period 1 and selects the one with the highest WSANPV. A strand is available for selection if it has no unfulfilled precursors. In this case, the only available strands are those starting with AEs 1 or 2 or MMF G. As Table 6 shows, the strand with the highest WSANPV for Pe-

riod 1 is **1A.**, at \$395K. So, AE 1 is developed in Period 1. Once AE 1 is developed, we can consider an additional set of strands starting with MMFs A, C, and D. The heuristic therefore adds these strands to the list of available strands for consideration in Period 2 and removes all strands that start with AE 1.

For Period 2, the heuristic selects MMF A, with a WSANPV of \$645K. Because MMF A takes two periods to develop and because in this example we aren't allowing concurrent development, we skip Period 3 and consider the available strands for Period 4. This time, the heuristic selects MMF G. This process continues until the heuristic has sequenced all MMFs, the project has exhausted the available development time, or no remaining strands have a positive SANPV. In this example, the heuristic selects sequence **1A.CGD2EFB**.

To calculate the project's NPV, we sum the SANPVs for each selected element. As Table 7 shows, this project has these characteristics:

- It generates just over \$1.6M.
- It requires \$350K of investment and becomes self-funding in Period 4.
- It's cash-positive in Period 10.
- At a discount rate of 2.5 percent per quarter, it has an NPV of just over \$1.1M.
- In DCF terms, it reaches breakeven in Period 11 (NPV > 0).

A thorough computational analysis indicates that the identified sequence returns 98.6 percent of the optimal NPV. The optimal sequence is **1A.C2EFGDB**, with an NPV of \$1.126M.

Analyzing the heuristic. In an empirical study, we tested 100 sets each of 10 and 12 elements at weighting factors of 0, 0.05, 0.10, 0.15, 0.20, and 0.25. For the elements, the per-period costs ranged from \$50K to \$200K. Development lasted from one to three periods. Revenues started at \$30K to \$100K and increased at a flat, linear, or exponential rate, which we also assigned within a validated range. We randomly generated dependencies within the precursor tree according to probabilities of depth and number of children and validated them through actual case studies. Following each run, we compared the IFM sequence's NPV with the optimal value obtained through brute-force analysis. This

Table 7

A financial analysis of linear sequencing for the online travel agency example (US\$ thousands). Self-funding occurs in Period 4, the project becomes cash positive in Period 10, and breakeven occurs in Period 11.

Element	Period																Net
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	
AE 1	-200	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	-200
MMF A		-75	-75	60	65	70	75	80	85	90	95	100	105	110	115	120	1,020
MMF C				-50	13	15	17	19	21	23	25	27	29	31	33	35	238
MMF G					-75	25	25	25	25	25	25	25	25	25	25	25	200
MMF D						-50	10	12	14	16	18	20	22	24	26	28	140
AE 2							-110	0	0	0	0	0	0	0	0	0	-110
MMF E								-80	25	28	30	33	37	40	44	49	206
MMF F									-75	15	20	25	30	35	35	35	120
MMF B										-50	6	9	10	11	11	12	9
Cash	-200	-75	-75	10	3	60	17	56	95	147	219	239	258	276	289	304	1,623
Net cash	-200	-275	-350	-340	-337	-277	-260	-204	-109	38	257	496	754	1,030	1,319	1,623	
Investment	-200	-75	-75														350
DCF @ 2.5%	-195	-71	-70	9	3	52	14	46	76	115	167	178	187	195	200	205	1,110
NPV	-195	-267	-336	-327	-324	-273	-258	-212	-136	-22	145	323	510	706	905	1,110	

Table 8

A financial analysis of concurrent sequencing for the online travel agency example (US\$ thousands). Self-funding occurs in Period 5, the project becomes cash positive in Period 8, and breakeven occurs in Period 9.

Element	Period																Net
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	
AE 1	-200	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	-200
MMF A		-75	-75	60	65	70	75	80	85	90	95	100	105	110	115	120	1,020
MMF C				-50	13	15	17	19	21	23	25	27	29	31	33	35	238
MMF B					-50	6	9	10	11	11	12	13	14	14	15	16	81
AE 2	-110	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	-110
MMF E		-80	25	28	30	33	37	40	44	49	54	59	65	71	78	86	619
MMF F			-75	15	20	25	30	35	35	35	35	35	35	35	35	35	330
MMF G				-75	25	25	25	25	25	25	25	25	25	25	25	25	225
MMF D					-50	10	12	14	16	18	20	22	24	26	28	30	170
Cash	-310	-155	-125	-22	53	184	205	223	237	251	266	281	297	312	329	347	2,373
Net cash	-310	-465	-590	-612	-559	-375	-170	53	290	541	807	1,088	1,385	1,697	2,026	2,373	
Investment	-310	-155	-125	-22													-612
DCF @ 2.5%	-302	-148	-116	-20	47	159	172	183	190	196	203	209	215	221	227	234	1,670
NPV	-302	-450	-566	-586	-539	-380	-208	-25	165	361	564	773	988	1,209	1,436	1,670	

study identified 0.15 as the optimal weighting factor for projects of 10 and 12 MMFs; at this weighting, the IFM heuristic returned an average of 98.5 percent of optimal NPV.

Comparison to other techniques

Several different methods exist for priori-

tizing requirements. Joachim Karlsson and Kevin Ryan discussed a cost-value approach that applied the Analytic Hierarchy Process to compare the cost and value ratios of requirements.¹¹ However, this method dealt only with comparisons between requirements, rather than actual cost and revenue projections, and

**In IFM,
financially
informed
decision making
is integral to
requirements
prioritization.**

therefore provided a much coarser-grained analysis than IFM.

Other researchers have considered multiple prioritization factors. For example, Guenther Ruhe and Desmond Greer developed EVOLVE+, a method that used genetic algorithms to assign requirements to releases according to user priorities; risk factors; development effort; and precedence, coupling, and resource constraints.¹² However they didn't explicitly incorporate revenue projections into the model, so it lacked the level of financial analysis that IFM provides. The primary benefit of GA approaches is their suitability for finding solutions in large search spaces, so in this respect they're ideally suited to solving prioritization problems. However, they're more complex to implement and take longer to converge on a solution.

The IFM approach is actually a special case of a linear-programming problem in which a complex problem must be solved through maximizing an objective such as cost or value. However, IFM's complexity increases because the value of the elements being scheduled changes according to their position in the schedule. IFM's strengths are that it's simple to implement and that you can quickly obtain a good solution with only the support of a basic spreadsheet. IFM also scales well for larger systems because the heuristic analyzes strands (which are limited in number) rather than all feasible sequences. Furthermore, in extremely large projects, you can use a windowing technique to defer consideration of MMFs that aren't of immediate interest.²

The impact of concurrent development

You can use IFM to identify concurrent schedules by simply selecting multiple elements for any given period. Space constraints prohibit a step-by-step example illustrating the selection of a concurrent sequence. However, assuming that concurrency between dependent elements isn't feasible and that only two elements may be developed per period, the IFM heuristic would select AEs 1 and 2 in Period 1, MMFs A and E in Period 2, only MMF F in Period 3 (because MMF A takes two periods), MMFs G and C in Period 4, and MMFs B and D in Period 5. The resulting concurrent sequence (see Table 8) exhibits these characteristics:

- Its NPV is \$1.67M, versus \$1.11M for the linear sequence.
- It requires \$612K to fund, versus \$350K for the linear sequence.
- The project reaches self-funding in Period 5 and breakeven in Period 9, versus Periods 4 and 11 for the linear sequence.

These results illustrate some of the trade-offs that exist between concurrent and sequential development. A concurrent schedule might be attractive because it enables the development of more features in less time and because it tends to increase NPV. However, it might require a higher initial investment than the organization can fund.

You can sometimes improve a project's fundability by considering a hybrid solution that gradually ramps up the project and introduces concurrency once earlier MMFs have started to generate revenue. This can decrease the initial funding requirement and return NPV somewhere within the range of the linear and fully concurrent solutions.

If a project needs both to maintain a lower level of funding and to attempt to reach self-funding status sooner, you can promote MMFs that will return value more quickly. In general, the IFM heuristic optimizes value over the entire analysis window; however, for sequencing purposes only, you can shorten the number of periods over which each element is analyzed. This causes the heuristic to favor elements that immediately start generating revenue following delivery, and ultimately brings forward the self-funding point.

In the IFM software development approach, financially informed decision making is integral to requirements prioritization. Only by recognizing that development decisions critically affect a project's financial well-being can organizations create a value-based software engineering environment that supports the success of their development endeavors.² All organizations, whether large or small, agile or traditional, have the capability to start thinking this way.

Furthermore, as businesses face pressure to focus on their core competencies, they're increasingly looking to outsourcing as a viable option. This introduces the challenge of finding better pricing and payment options. IFM

deconstructs an application into deliverable features that not only reduce the risks of large-scale development but also provide a natural framework for establishing staged payments. This is beneficial to both the systems' integrator and the customer.

This model can extend to application service providers. IFM equips the ASP with the information needed to determine a cost structure that's beneficial to all parties involved. Both the ASP and the customer clearly benefit from the enhanced visibility into the project's financial and technical progress. IFM's incremental nature provides early warning when projects start to go wrong, enabling midcourse corrections and preventing the kinds of project death marches described by Ed Yourdon¹³ and documented in the Chaos Report.⁷

A spreadsheet illustrating the example described in this article is available at www.softwarebynumbers.org. 

References

1. B. Boehm, "Value-Based Software Engineering," *ACM SIGSOFT Software Eng. Notes*, vol. 28, no. 2, Mar. 2003, p. 4.
2. M. Denne and J. Cleland-Huang, *Software by Numbers: Low-Risk, High-Return Development*, Prentice Hall, 2003.
3. T. Gilb, *Principles of Software Engineering Management*, Addison-Wesley, 1988.
4. K. Beck, *Extreme Programming Explained: Embrace Change*, Addison-Wesley, 1999.
5. H. Erdogmus and J. Favaro, "Keep Your Options Open: Extreme Programming and the Economics of Flexibility," *Extreme Programming Perspectives*, G. Succi et al., eds., Addison-Wesley, 2002, pp. 503-552.
6. P. Kroll and P. Kruchten, *The Rational Unified Process Made Easy: A Practitioner's Guide to the RUP*, Addison-Wesley, 2003.
7. *The Chaos Report*, Standish Group, 1995.
8. L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, Addison-Wesley, 1997.
9. M. Shaw and D. Garlan, *Software Architecture: Perspectives on an Emerging Discipline*, Prentice Hall, 1996.
10. K. Sullivan et al., "Software Design as an Investment Activity: A Real Options Perspective," *Real Options and Business Strategy: Applications to Decision Making*, L. Trigeorgis, ed., Risk Books, 1999.
11. J. Karlson and K. Ryan, "A Cost-Value Approach for Prioritizing Requirements," *IEEE Software*, vol. 14, no. 5, 1997, pp. 67-74.
12. G. Ruhe and D. Greer, "Quantitative Studies in Software Release Planning under Risk and Resource Constraints," *Proc. IEEE Int'l Symp. Empirical Software Eng.*, IEEE CS Press, 2003, pp. 262-271.
13. E. Yourdon, *Death March*, Prentice Hall, 1999.

For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/publications/dlib

About the Authors



Mark Denne is the US Edge Solutions Manager for Sun Microsystems, where he oversees the creation of business and infrastructure solutions for Grid computing. He previously led the Sun Java Center in New York City, managing architects and designers working with financial services and media clients. He received his master's in computer science from Cambridge University. He's the coauthor (along with Jane Cleland-Huang) of *Software by Numbers* (Prentice Hall, 2003). Contact him at 100 California St., Ste. 800, San Francisco, CA 94111; mark.denne@sun.com.

Jane Cleland-Huang is an assistant professor at DePaul University's School of Computer Science, Telecommunications, and Information Systems. She also oversees the DePaul Center for Applied Requirements Engineering and supervises an NSF-funded research project on requirements traceability. Her research interests include process models, requirements engineering, and value-based software engineering. She received her PhD in computer science from the University of Illinois at Chicago. Contact her at the School of Computer Science, Telecommunications, and Information Systems, DePaul Univ., 342 S. Wabash, Chicago, IL 60604; jhuang@cs.depaul.edu.



Classified Advertising

SALES ENGINEER - Must have BS in Electrical Engineering and 6 years of experience in the job offered or as an account/territory manager. Responsible for developing new business with a focus on design and delivery of integrated solutions that require the application of principals of engineering and software design and development including needs assessment, ongoing technical support and relationship management. Must be able to respond to RFI's and RFP's including preparation of cost estimates and ROI statements for clients. Qualified candidates must send resume to Vector Consulting, Attn: Sanja Agrawala, V.P., 6455 East Johns Crossing, Suite 450, Duluth, Georgia 30097.



SENIOR APPLICATION SPECIALIST (SCT BANNER STUDENT MODULE) - Implement and perform gap/fit analysis of SCT Banner Student module for Admissions, Records, Financial Aid, and other student business units. Oversee development of and write technical specifications for add-on/bolt-on applications. Analyze and develop interfaces from the third party systems. Design, configure and support externally purchased ERP system. Review existing Legacy systems and map and convert Legacy application data into the SCT Banner Student module. Create web applications using Java, ColdFusion, and SQL. Work with teams on systems unit, and on functional, regression, and performance testing. Must have MS or foreign equivalent with a minimum of 15 credit hours in Computer Science, M.I.S. or related and 2 yrs. experience with SCT Banner student

module at a major research university. Must have authorization to work in the U.S. on a permanent basis. 40-hrs/wk, 8A to 5P, M-F. Qualified applicants send resumes to University of Notre Dame, Attn: C. Berzai, 235 Info Tech Ctr., Notre Dame, IN 46556.



SENIOR PROCUREMENT ANALYST: Design, develop, and implement e-procurement software solutions in conjunction with external partners for business operations. Implement and manage supplier integration and e-invoicing projects for e-procurement including defining business and software rules, testing applications and developing documentation related to the project. Serve as an expert in using information systems required to execute purchasing transactions. Using Rapid Product Development methods, develop product roadmap in collaboration with business partners and internal departments. Using "Fit and Gap" techniques, analyze software requirements to determine feasibility of design within resource and business constraints. Analyze e-sourcing tools to develop Request for Proposal documents. Perform data analysis on purchasing data and report to executive members. Use best practices, information architecture skills, and advanced web development tools to develop the department website. Assist in the design and execution of marketing initiatives, including newsletters and other electronic marketing media. Must have MBA, 2 yrs. exp. in job offered or 2 yrs. exp. as a Data Management Analyst, and authorization to work in the U.S. on a permanent basis. 40-hrs/wk, 8A to 5P. Qualified applicants send résumés to University of Notre Dame, Attn: L. Kish, 709 Grace Hall, Notre Dame, IN 46556.